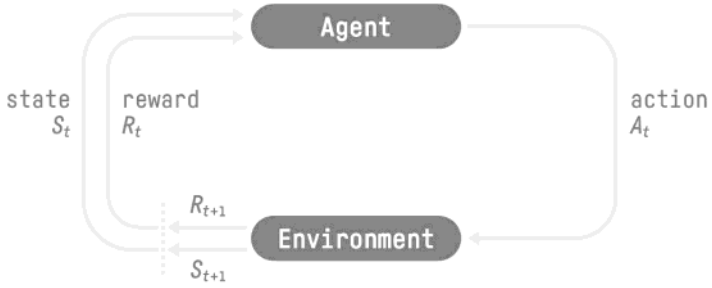
Chapter 1 From Huggingface Deep RL Course Introduction to Deep Reinforcement Learning

02 May 2024 13:33

1. What is Reinforcement Learning is just a computational approach of learning from actions. Reinforcement learning is a framework for solving control tasks (also called decision problems) by building agents that learn from the environment by interacting with it through trial and error and receiving rewards (positive or negative) as unique feedback.

2. The Reinforcement Learning framework
   a. RL Proccess



- Our Agent receives **state S0S0** from the **Environment** — we receive the first frame of our game (Environment).
- Based on that **stateS0S0,** the Agent takes **actionA0A0** — our Agent will move to the right.
- The environment goes to a **new stateS1S1** — new frame.
- The environment gives some **rewardR1R1** to the Agent — we're not dead *(Positive Reward +1)*.

This RL loop outputs a sequence of **state, action, reward and next state.**

1. The reward hypothesis: the central idea of Reinforcement Learning

   Because RL is based on the reward hypothesis, which is that all goals can be described as the maximization of the expected return (expected cumulative reward).

   That's why in Reinforcement Learning, to have the best behavior, we aim to learn to take actions that maximize the expected cumulative reward.

2. Markov property

   Markov Property implies that our agent needs only the current state to decide what action to take and not the history of all the states and actions they took before.

3. Observations/States Space

   Observations/States are the information our agent gets from the environment. In the case of a video game, it can be a frame (a screenshot). In the case of the trading agent, it can be the value of a certain stock, etc. In a chess game, we have access to the whole board information, so we receive a state from the environment. In other words, the environment is fully observed. In Super Mario Bros, we only see the part of the level close to the player, so we receive an observation.

4. Action Space
   In Super Mario Bros, we have only 4 possible actions: left, right, up (jumping) and down (crouching). A Self Driving Car agent has an infinite number of possible actions since it can turn left 20°, 21,1°, 21,2°, honk, turn right 20°…

Taking this information into consideration is crucial because it will have importance when choosing the RL algorithm in the future.

5. Reward and Discounting

The reward is fundamental in RL because it's the only feedback for the agent. Thanks to it, our agent knows if the action taken was good or not. The cumulative reward at each time step t can be written as:

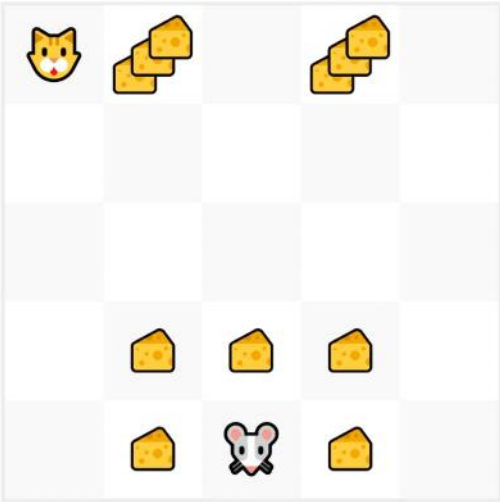
$$R(\tau) = r_{t+1} + r_{t+2} + r_{t+3} + r_{t+4} + \dots$$

Return: cumulative reward

Trajectory (read Tau)
Sequence of states and actions

$$R(\tau) = \sum_{k=0}^{\infty} r_{t+k+1}$$

However, in reality, we can't just add them like that. The rewards that come sooner (at the beginning of the game) are more likely to happen since they are more predictable than the long-term future reward. Let's say your agent is this tiny mouse that can move one tile each time step, and your opponent is the cat (that can move too). The mouse's goal is to eat the maximum amount of cheese before being eaten by the cat.



As we can see in the diagram, it's more probable to eat the cheese near us than the cheese close to the cat (the closer we are to the cat, the more dangerous it is). Consequently, the reward near the cat, even if it is bigger (more cheese), will be more discounted since we're not really sure we'll be able to eat it. To discount the rewards, we proceed like this: We define a discount rate called gamma. It must be between 0 and 1. Most of the time between 0.95 and 0.99. The larger the gamma, the smaller the discount. This means our agent cares more about the long-term reward. On the other hand, the smaller the gamma, the bigger the discount. This means our agent cares more about the short term reward (the nearest cheese). Then, each reward will be discounted by gamma to the exponent of the time step. As the time step increases, the cat gets closer to us, so the future reward is less and less likely to happen. Our discounted expected cumulative reward is:

$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$

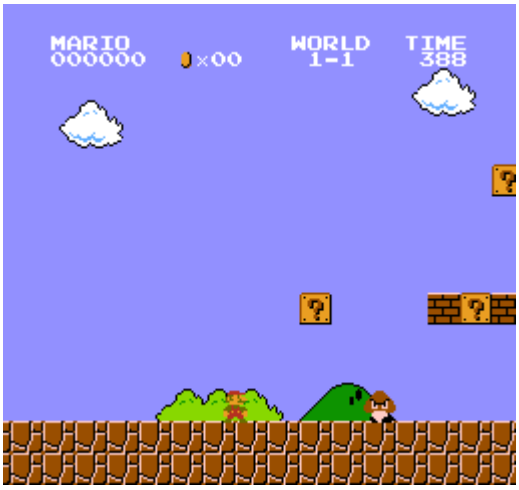
Return: cumulative reward

Gamma: discount rate

Trajectory (read Tau)
Sequence of states and actions

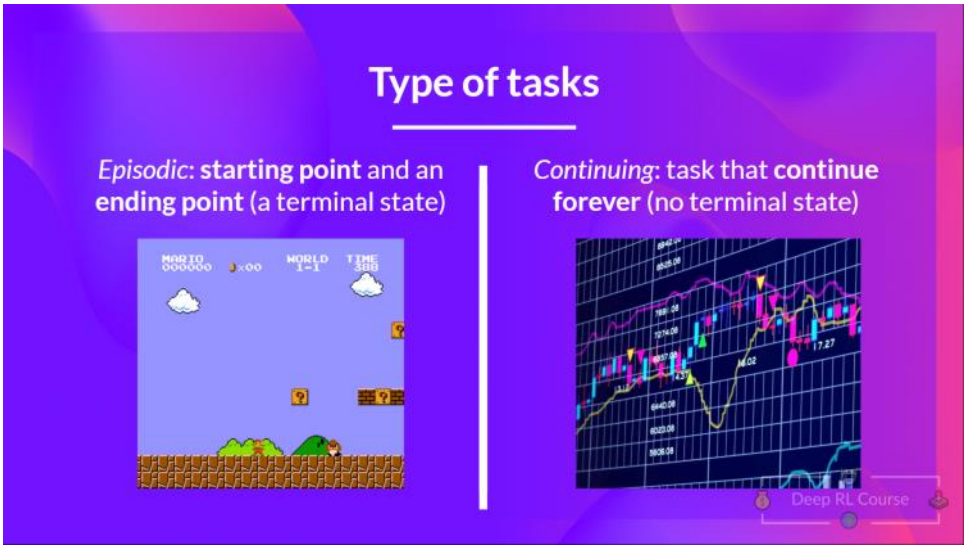
$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

6. Type of Tasks
   a. Episodic Tasks
      In this case, we have a starting point and an ending point (a terminal state). This creates an episode: a list of States, Actions, Rewards, and new States. For instance, think about Super Mario Bros: an episode begin at the launch of a new Mario Level and ends when you're killed or you reached the end of the level.



b. Continuing Tasks

These are tasks that continue forever (no terminal state). In this case, the agent must learn how to choose the best actions and simultaneously interact with the environment. For instance, an agent that does automated stock trading. For this task, there is no starting point and terminal state. The agent keeps running until we decide to stop it.



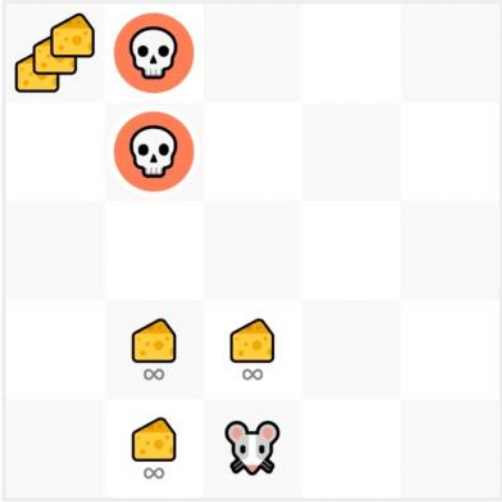
7. The Exploration/Exploitation trade-off

Finally, before looking at the different methods to solve Reinforcement Learning problems, we must cover one more very important topic: the exploration/exploitation trade-off.

a. Exploration is exploring the environment by trying random actions in order to find more information about the environment. Exploitation is exploiting known information to maximize the reward.
b. Remember, the goal of our RL agent is to maximize the expected cumulative reward. However, we can fall into a common trap.

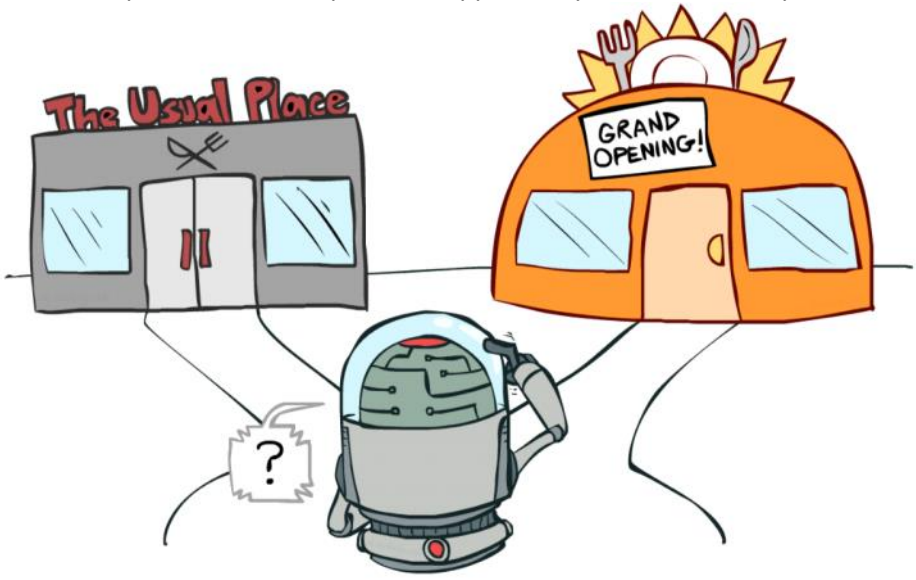
Let's take an example:

a. Mouse and Cheese

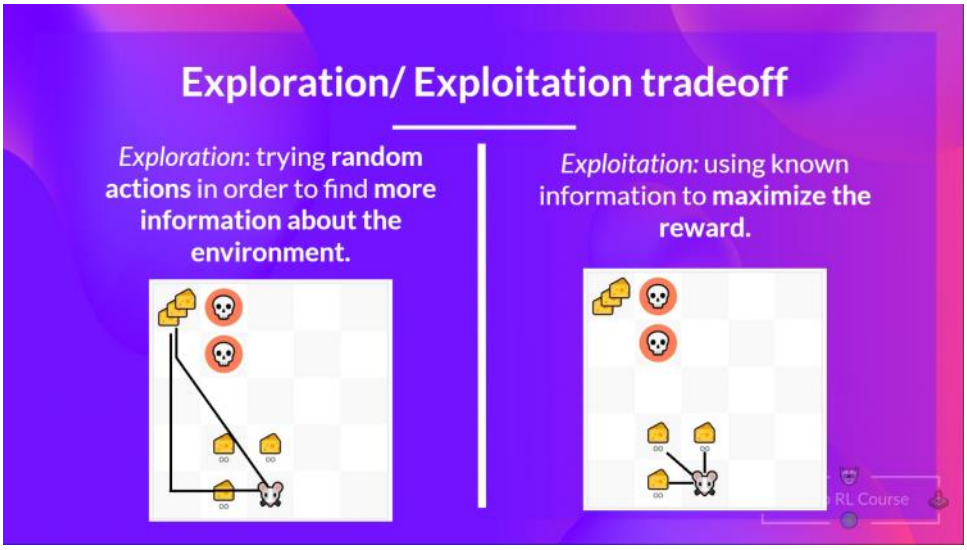


In this game, our mouse can have an infinite amount of small cheese (+1 each). But at the top of the maze, there is a gigantic sum of cheese (+1000). However, if we only focus on exploitation, our agent will never reach the gigantic sum of cheese. Instead, it will only exploit the nearest source of rewards, even if this source is small (exploitation). But if our agent does a little bit of exploration, it can discover the big reward (the pile of big cheese).This is what we call the exploration/exploitation trade-off. We need to balance how much we explore the environment and how much we exploit what we know about the environment. Therefore, we must define a rule that helps to handle this trade-off. We'll see the different ways to handle it in the future units.

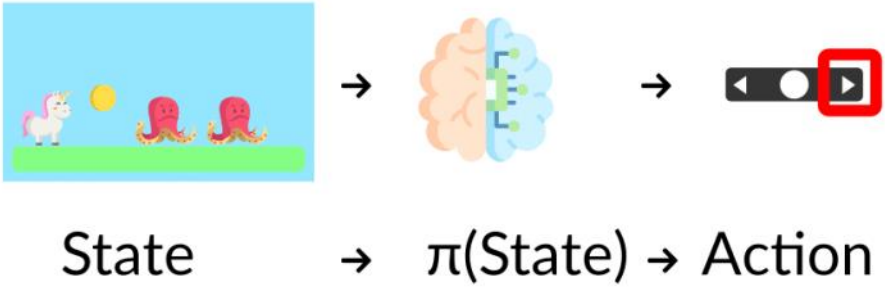
b. Picking a restaurant:
    i. Exploitation You go to the same one that you know is good every day and take the risk to miss another better restaurant.
    ii. Exploration: Try restaurants you never went to before, with the risk of having a bad experience but the probable opportunity of a fantastic experience.



c. Exploitation vs Exploration illustration

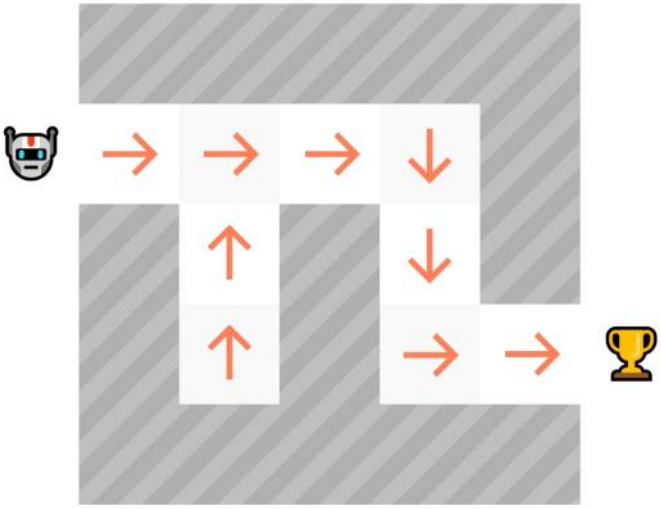


8. Two main approaches for solving RL problems
   In other words, how do we build an RL agent that can select the actions that maximize its expected cumulative reward?
   a. The Policy π: the agent's brain
      The Policy π is the brain of our Agent, it's the function that tells us what action to take given the state we are in. So it defines the agent's behavior at a given time. Think of policy as the brain of our agent, the function that will tell us the action to take given a state



      This Policy is the function we want to learn, our goal is to find the optimal policy π*, the policy that maximizes expected return when the agent acts according to it. We find this π* through training.
      There are two approaches to train our agent to find this optimal policy π*:
      i. Directly, by teaching the agent to learn which action to take, given the current state: Policy-Based Methods.
      ii. Indirectly, teach the agent to learn which state is more valuable and then take the action that leads to the more valuable states: Value-Based Methods.

   b. Policy-Based Method
      In Policy-Based methods, we learn a policy function directly. This function will define a mapping from each state to the best corresponding action. Alternatively, it could define a probability distribution over the set of possible actions at that state.
      Policy



i. Deterministic: a policy at a given state will always return the same action.
action = policy(state)

$$a = \pi(s)$$

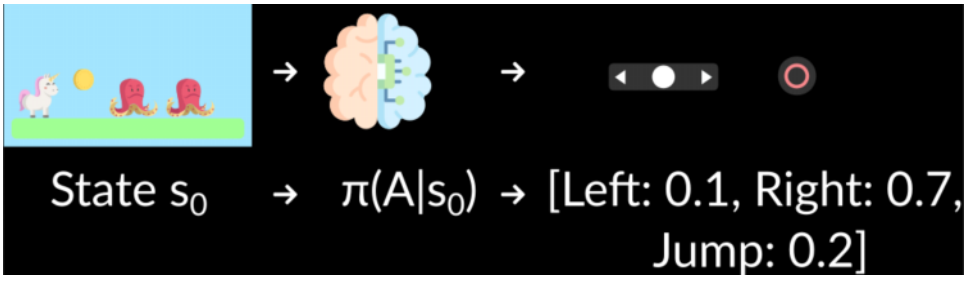
State s_0 → $\pi(s_0)$ → a_0 = Right

ii. Stochastic: a probability distribution over actions.

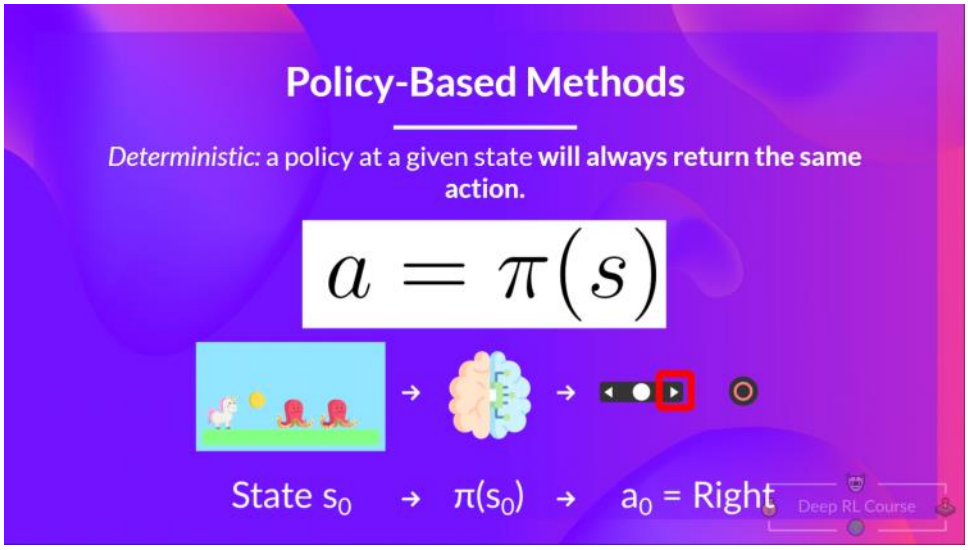
$$\pi(a|s) = \boxed{P[A|s]}$$

Probability Distribution over the set of actions given the state

policy(actions | state) = probability distribution over the set of actions given the current state



i. Deterministic vs Stochastic

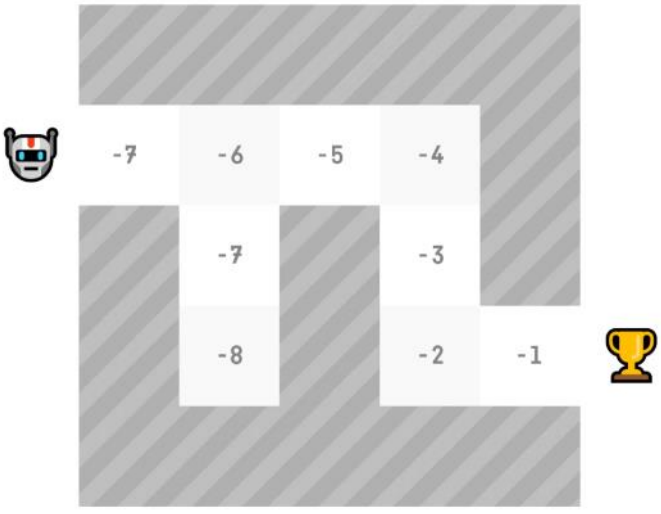


c. Value-Based Method

In value-based methods, instead of learning a policy function, we learn a value function that maps a state to the expected value of being at that state. The value of a state is the expected discounted return the agent can get if it starts in that state, and then acts according to our policy. “Act according to our policy” just means that our policy is “going to the state with the highest value”.

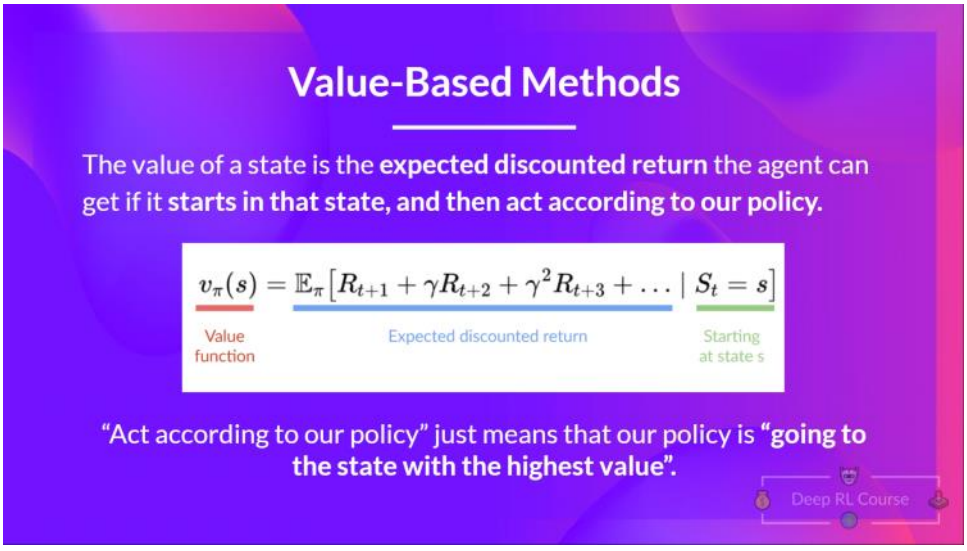
$$\underbrace{v_\pi(s)}_{\text{Value function}} = \underbrace{\mathbb{E}_\pi\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots\right.}_{\text{Expected discounted return}} \left. \mid \underbrace{S_t = s}_{\text{Starting at state s}}\right]$$

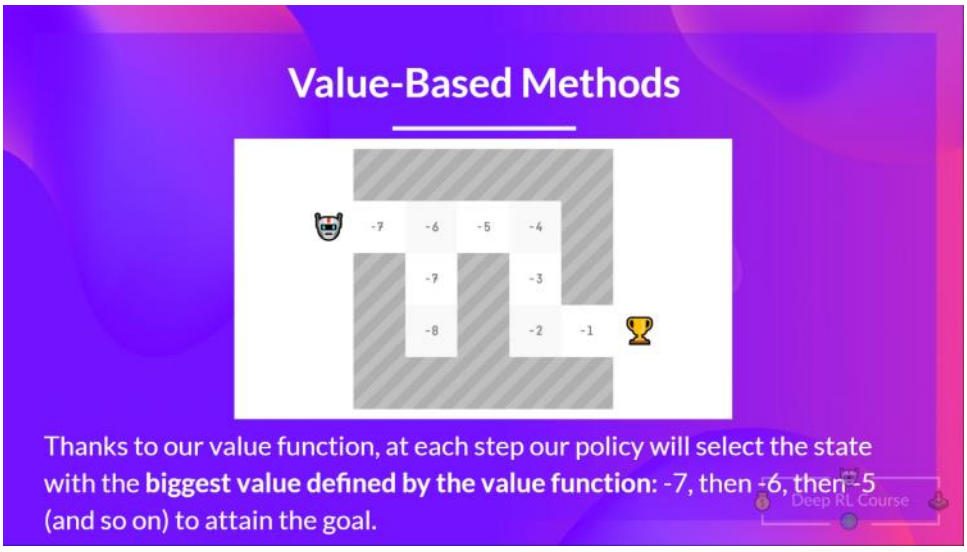
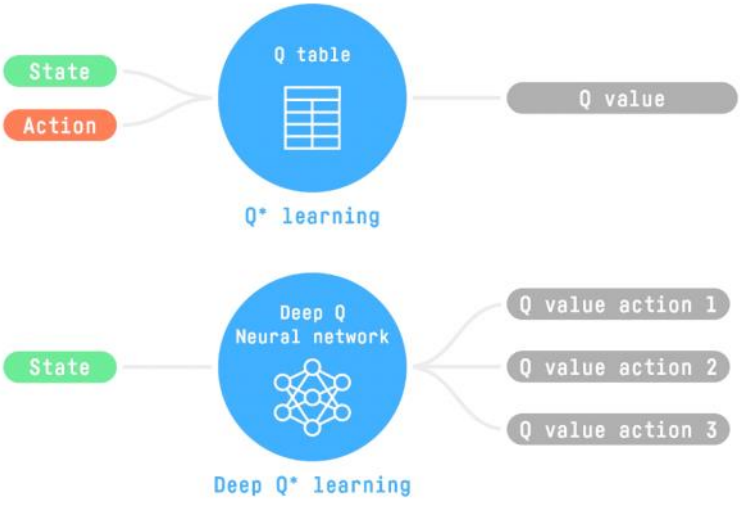
Here we see that our value function defined values for each possible state.



Thanks to our value function, at each step our policy will select the state with the biggest value defined by the value function: -7, then -6, then -5 (and so on) to attain the goal.

d. Policy-based method vs value-based method





9. The "deep" in the deep reinforcement learning
   Deep Reinforcement Learning introduces deep neural networks to solve Reinforcement Learning problems — hence the name "deep". For instance, in the next unit, we'll learn about two value-based algorithms: Q-Learning (classic Reinforcement Learning) and then Deep Q-Learning. You'll see the difference is that, in the first approach, we use a traditional algorithm to create a Q table that helps us find what action to take for each state. In the second approach, we will use a Neural Network (to approximate the Q value).